

AI BlogNest - Generative AI-Powered Blog Management Platform

Project Name	AI BlogNest API
Team ID	SWTID-2026-3342
Team Size	4
Team Leader	Sivaraj S (sivaraj0827@gmail.com)
Team Members	Veron Joseph Dcouto (veronjosephalt2006@gmail.com) · Alfred Vishal A (alfredvishal25@gmail.com) · Navin P (teamxnavin16@gmail.com)
Documentation Period	15 September 2026 - 29 September 2026

Project Description: AI BlogNest provides a centralized and secure blogging platform that combines traditional blog management with Generative AI. Users can generate blog content using AI, summarize existing posts, manage blogs, organize content with categories and tags, and handle reader comments. Role-based access allows Admins, Editors, Authors, and Readers to access features according to their responsibilities.

Scenario-Based Case Study

Background

Priya is a content creator who manages multiple blogs but finds it difficult to create, organize, summarize, and manage blog content efficiently. She spends significant time writing content and maintaining posts, categories, tags, and reader interactions.

Problem

- Difficulty creating high-quality blog content quickly.
- Time-consuming process of writing and editing blogs.
- Difficulty organizing posts using categories and tags.
- Limited support for generating content automatically.
- Time-consuming process of summarizing long blog posts.
- Difficulty managing comments and user roles.

Solution

AI BlogNest provides a centralized and secure blogging platform that combines traditional blog management with Generative AI. Users can generate blog content using AI, summarize existing posts, manage blogs, organize content with categories and tags, and handle reader comments. Role-based access allows Admins, Editors, Authors, and Readers to access features according to their responsibilities.

Usage

- **Author Usage:** Creates new blog posts using manual writing or AI-generated content, generates content with the AI Blog Generator, organizes blogs using categories and tags, edits/saves/manages drafts, and uses AI summarization to create concise summaries.
- **Editor/Admin Usage:** Reviews and manages submitted blog content, changes publishing status, manages categories and tags, moderates user comments, manages user roles and permissions, and accesses protected administrative endpoints.
- **Reader Usage:** Views and searches published blog posts, reads blog content and AI-generated summaries, adds comments to blog posts, and interacts with published content.

Outcome

AI BlogNest provides a centralized platform for creating and managing blog content with the support of Generative AI. AI-assisted content generation reduces the time required to create blog posts, while AI summarization helps users quickly understand lengthy content. Role-based access and security controls ensure that users can access only the features permitted for their roles. MongoDB maintains the application's blog, user, and interaction data.

1. Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js: Runtime environment used to run the backend server.
- npm: Package manager used to install and manage project dependencies.
- Express.js: Backend framework used to create RESTful APIs and manage server routes.
- MongoDB: NoSQL database used to store users, blogs, comments, categories, and tags.
- Mongoose: ODM used for defining schemas and interacting with MongoDB.
- Google Gemini AI: AI service used for blog generation and summarization.
- Postman: Used for testing and verifying REST API endpoints.
- Visual Studio Code: Development environment used for writing and managing the project code.

2. Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent.
- RAM: Minimum 8 GB recommended.
- Storage: At least 1 GB of available space for the project, dependencies, and development files.

EPIC 1: Project Architecture

The AI BlogNest application uses a layered server architecture in which incoming requests pass through authentication, validation, and routing layers before interacting with business logic, AI services, and the database.

Components Definition

- **Interface:** Tools such as Postman and the frontend send requests to the REST API endpoints and receive responses in JSON format.
- **Express Server Gateway:** Handles incoming HTTP requests and maps methods such as GET, POST, PUT, and DELETE to the appropriate API routes. It also processes JSON data and manages CORS.
- **Authentication & Role Middleware:** Validates user authentication tokens and checks the user's role before allowing access to protected resources. Different permissions are provided for Admin, Editor, Author, and Reader roles.
- **Blog Management Services:** Handles the main blog operations such as creating, reading, updating, and deleting blogs. It also manages blog drafts, publishing status, categories, tags, and comments.
- **Database Interface Layer:** Mongoose provides the connection between the application and MongoDB, allowing the system to store and retrieve users, blogs, comments, categories, and tags.

- **AI Flow (Service Integration):** Provides AI-powered functionality for blog generation and blog summarization. The backend communicates with the Gemini AI service through dedicated endpoints such as `/api/ai/generate-blog` and `/api/ai/summarize`.
- **BlogGenerator with AI:** A specialized service that sends the user's topic or requirements to the AI service and generates structured blog content, helping users create blog posts more efficiently.

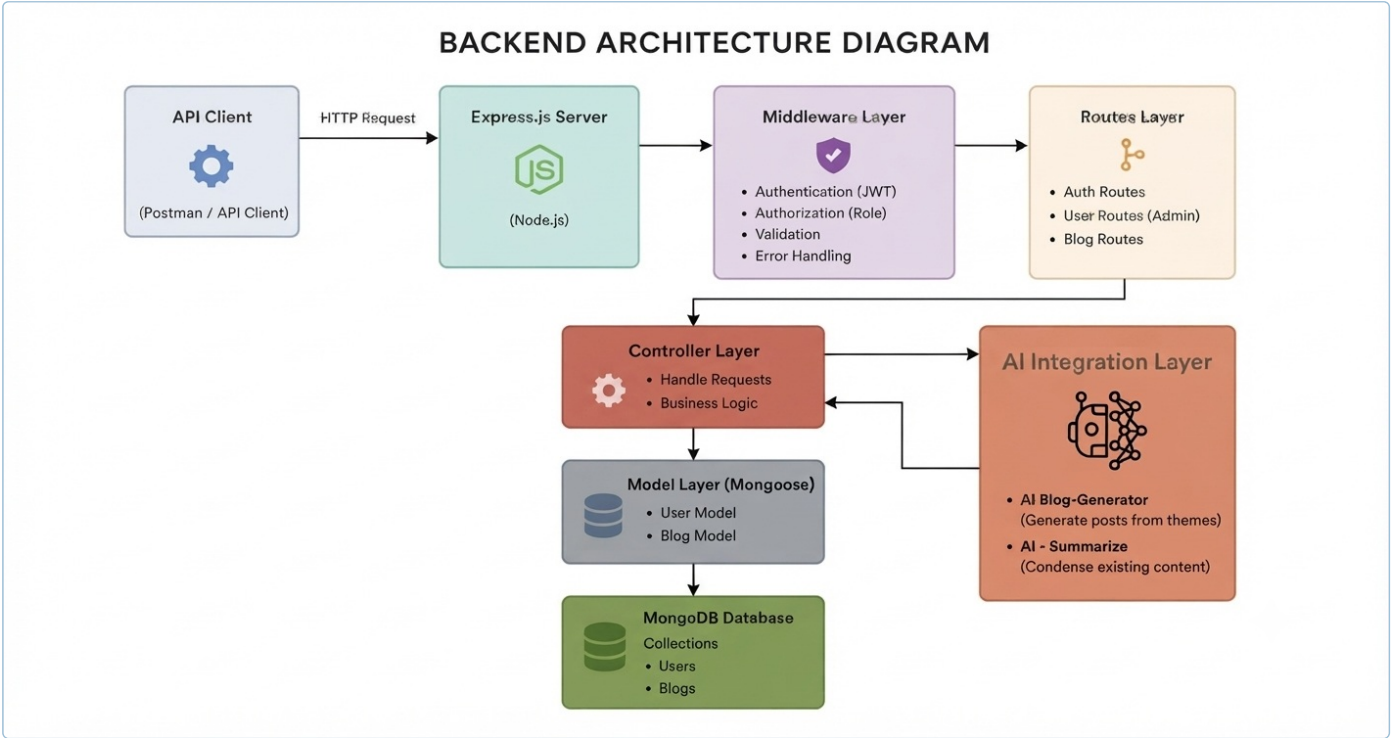


Fig. 1 – AI BlogNest backend architecture diagram: API Client → Express.js Server → Middleware Layer (Authentication/Authorization/Validation/Error Handling) → Routes Layer → Controller Layer ↔ AI Integration Layer (AI Blog-Generator, AI-Summarize) → Model Layer (Mongoose) → MongoDB Database.

Entity-Relationship (ER) Diagram Description

The underlying document structure manages several collections bound by logical application references:

1. User Entity

- `_id` — Primary Key (ObjectId)
- `name` — String, Required
- `email` — String, Required, Unique
- `password` — String, Hashed, Required
- `role` — String, Default: "reader"

2. Blog Entity

- `_id` — Primary Key (ObjectId)
- `userID` — Reference to User Collection
- `title` — String, Required
- `content` — String, Required
- `photo` — String, Optional image/path link
- `category` — String
- `tags` — Array of Strings
- `status` — String, Default: "draft"
- `likes` — Number, Default: 0
- `timestamps` — True

3. Comment Entity

- `_id` — Primary Key (ObjectId)
- `userID` — Reference to User Collection
- `blogID` — Reference to Blog Collection
- `comment` — String, Required
- `status` — String, Default: "pending"
- `timestamps` — True

Key Structural Mappings

- **User to Blog (One-to-Many):** One author can create and manage multiple blog posts.
- **User to Comment (One-to-Many):** A user can add multiple comments across different blog posts.
- **Blog to Comment (One-to-Many):** A single blog post can contain multiple comments from different users.
- **User to Role (One-to-One):** Each user has one assigned role such as Admin, Editor, Author, or Reader, which determines their access permissions.

BlogNest – ER Diagram

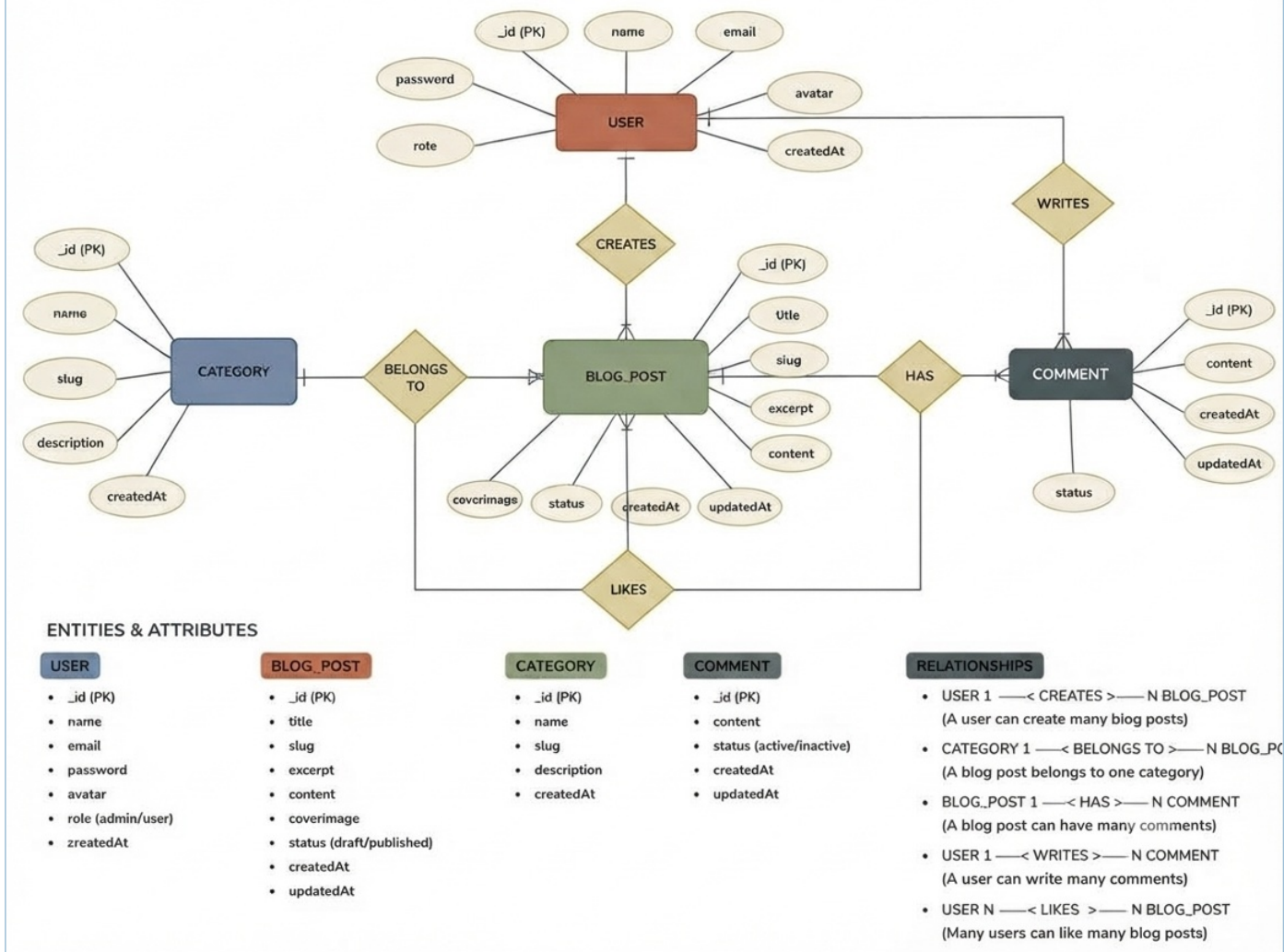


Fig. 2 – Extended ER design explored during architecture planning (USER, BLOG_POST, CATEGORY and COMMENT with CREATES/BELONGS TO/HAS/WRITES/LIKES relationships). The currently implemented model is the simpler User → Blog → Comment structure described above; the CATEGORY collection and the richer Blog fields (slug, excerpt, coverImage, a dedicated likes relation) shown here are carried forward as the Scalability & Future Plan roadmap.

Key Features

- **Role-Based Access Control:** JWT authentication and role-based permissions restrict access to protected features for Admin, Editor, Author, and Reader users.
- **CRUD Blog Management:** Allows authorized users to create, read, update, and delete blog posts while managing drafts and publishing status.
- **Categories and Tags:** Organizes blog posts using categories and tags, making content easier to filter and discover.
- **Comment Management:** Allows readers to comment on blogs and provides moderation options for managing comments.
- **Security and Validation:** Validates and sanitizes user input to help protect the application from malicious requests and data manipulation.
- **AI Blog Generation:** Uses Generative AI to create structured blog content from user-provided topics, keywords, or requirements.
- **AI Content Summarization:** Uses AI to analyze existing blog content and generate concise summaries for easier and faster reading.
- **REST API Integration:** Provides structured REST API endpoints that connect the frontend, backend, database, and AI services.

Roles and Responsibilities

Admin

- Manages registered users and their accounts.
- Assigns and manages user roles.
- Manages or removes inappropriate comments.
- Has access to administrative features and protected system resources.
- Monitors overall blog and user management activities.

Author

- Creates and manages their own blog posts.
- Uses AI tools to generate blog content.
- Creates and manages drafts.
- Edits and updates their own posts.
- Organizes their posts using categories and tags.

Editor

- Reviews submitted blog posts and drafts.
- Manages publishing status of blog content.
- Organizes and maintains categories and tags.
- Reviews content before publication.
- Helps maintain the quality and organization of published blogs.

Reader

- Views and reads published blog posts.
- Searches and explores available content.
- Reads AI-generated summaries where available.
- Posts comments on blog posts.
- Interacts with published content.

User Flow

AI-BLOG USER FLOW DIAGRAM (Backend Only)

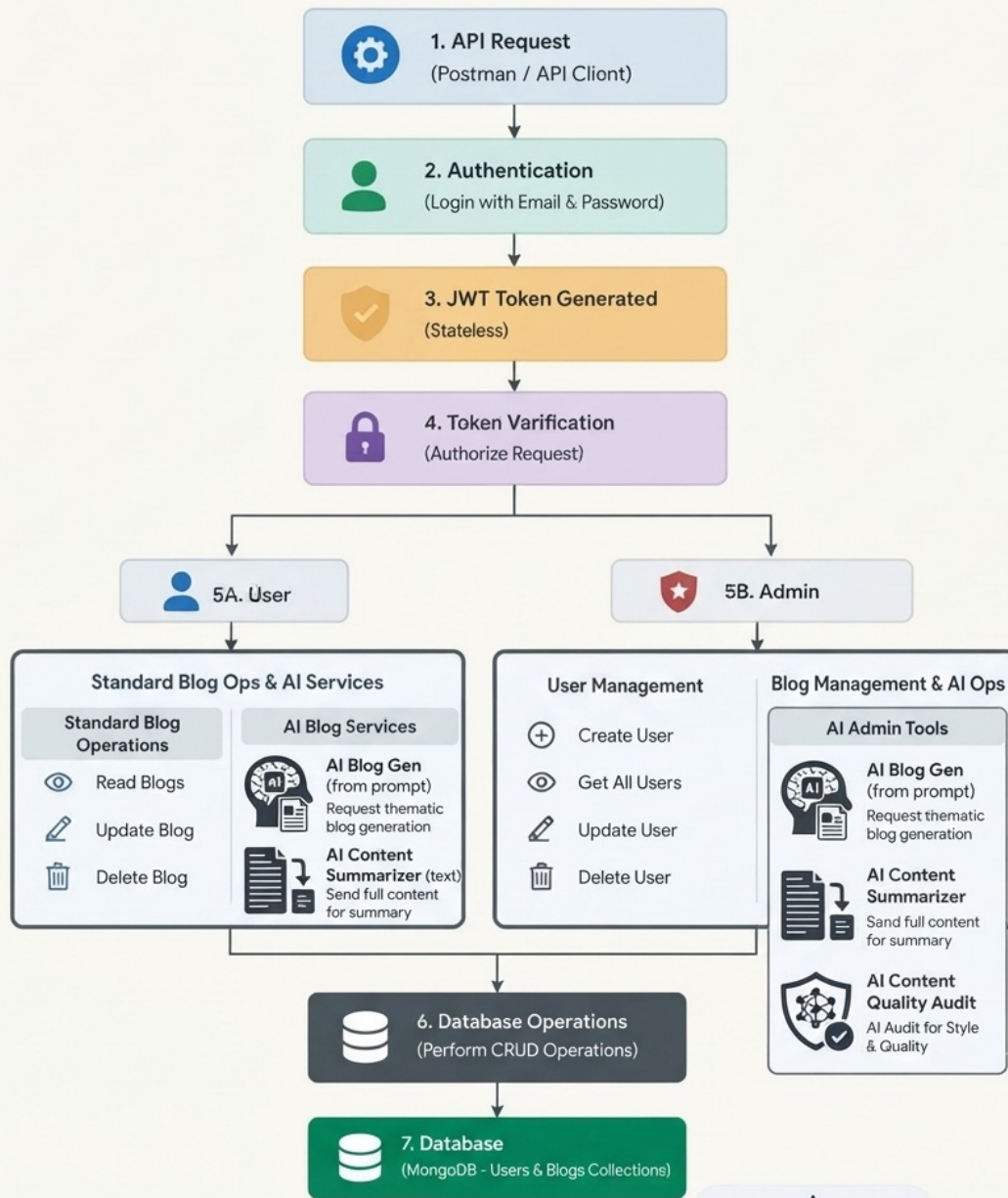


Fig. 3 – AI-Blog user flow (backend only): API Request → Authentication (email & password) → JWT Token generated (stateless) → Token verification → branches into User (standard blog ops + AI Blog Services) and Admin (user management + blog management & AI ops) → Database Operations (CRUD) → MongoDB (Users & Blogs collections).

MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

- Model Layer (Data Blueprint):** Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.
- Controller Layer (Execution Matrix):** The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.
- View Layer (Routing Network Interface):** In this pure headless backend service context, standard user-interface template views do not exist. Instead the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

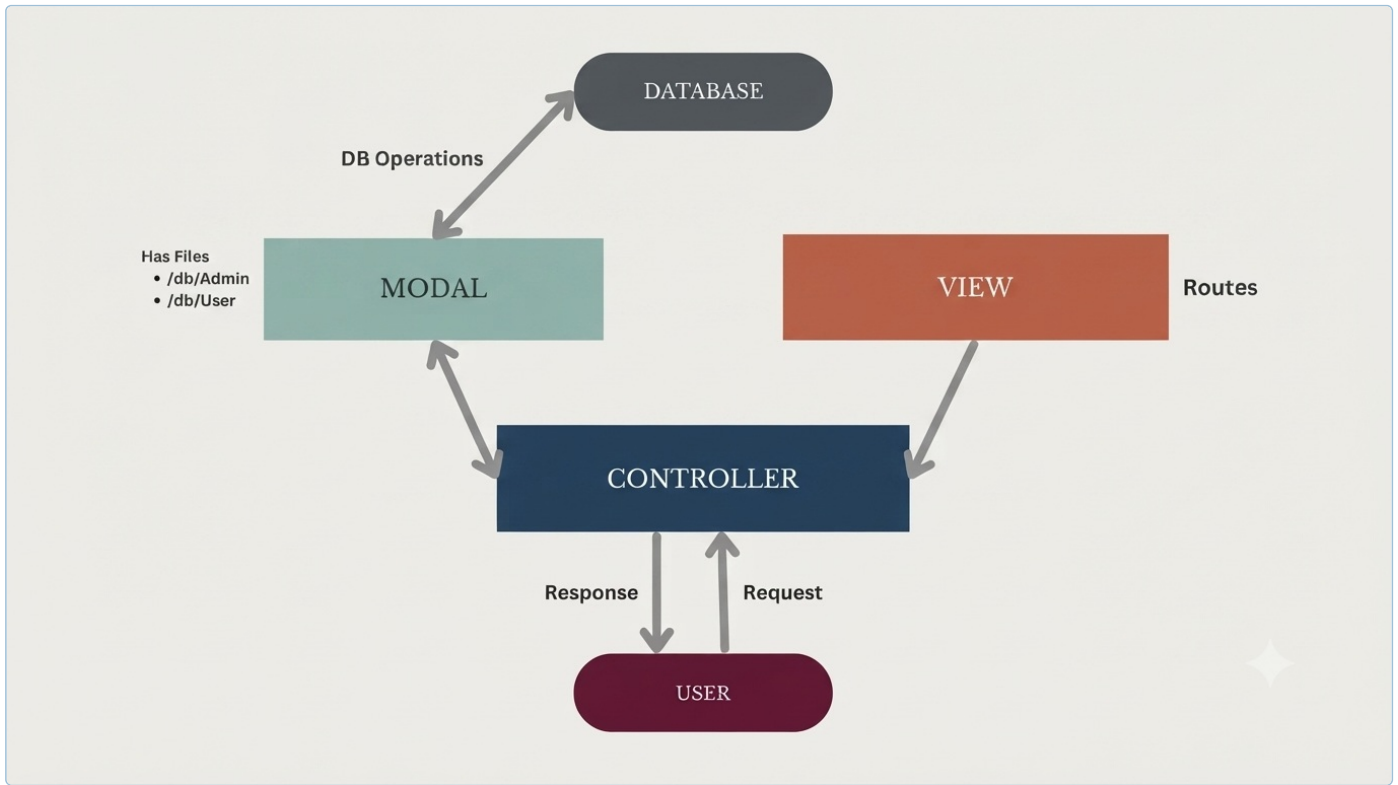


Fig. 4 – MVC pattern as applied in AI BlogNest: User ↔ Controller (Request/Response) ↔ Modal/Model (DB Operations, files under /db/Admin, /db/User) ↔ View (Routes) ↔ Database.

Epic 2: Project Setup and Configuration

Creating Project Folder

- Create a new folder named AI-BlogNest.
- Inside the project folder, create a folder named Client for the frontend files.
- Create another folder named Server for the backend files.
- Open the AI-BlogNest folder in Visual Studio Code.

Server Setup (npm init)

- Open the Server folder in the VS Code terminal.
- Initialize the Node.js project using: `npm init -y`
- Create the main server file: `server.js`
- Create the following folders inside the Server folder: `config`, `controllers`, `middleware`, `models`, `routes`, `services`

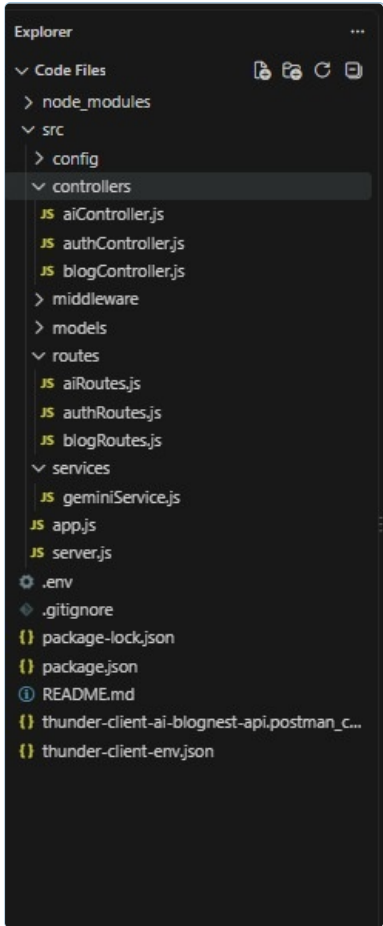


Fig. 5 – Final Server/ folder structure created for the project.

Environment Configuration

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
GEMINI_API_KEY=your_gemini_api_key
```

Epic 3: Backend Development

The AI BlogNest backend is developed using Node.js, Express.js and MongoDB, following a modular architecture. The application separates routing, controllers, middleware, models and service modules to improve maintainability and scalability. The backend exposes secure REST APIs for authentication, blog management, comment handling and AI-powered content generation.

App.js

The application file wires together the Express app: it registers `cors`, JSON body parsing and `morgan` request logging, mounts the `/api/auth`, `/api/blogs` and `/api/ai` route groups, registers the shared error-handling middleware, and exposes a root health-check route that reports "AI BlogNest API is running".

```
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   | res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
```

Fig. 5b – app.js: configures cors, JSON parsing and morgan logging, mounts the auth/blogs/ai route groups plus the error middleware, and exports the configured Express app.

server.js (Application Entry Point)

The entry-point file loads environment variables with `dotenv`, requires the configured Express app and the database connector, connects to MongoDB, and starts the server on the configured PORT (defaulting to 5000).

```
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10   | console.log(`Server running on port ${PORT}`);
11 });
12
```

Fig. 5c – server.js: loads dotenv, calls connectDB(), then starts the Express app listening on `process.env.PORT || 5000`.

Epic 4: Database Configuration

1. Configure MongoDB

Mongoose is used to connect the Node.js application with MongoDB and define database schemas and models. In the Server directory, run:

```
npm install mongoose
```

- Create database connection.
- Create Schemas & Models.

Create Database connection

Now, make sure the database is connected before performing any of the actions through the backend. The connection code checks that `MONGO_URI` is defined, connects via Mongoose, logs the connected host, and exits the process on any failure.

```

1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
23

```

Fig. 6 - config/db.js: validates that MONGO_URI is set, connects with mongoose.connect(), logs the connected host, and calls process.exit(1) on error.

Mongoose Schema Definitions - Identity Resource Document Schema (schemas/userModel.js)

Enforces field criteria validations on incoming database creations:

```

1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29

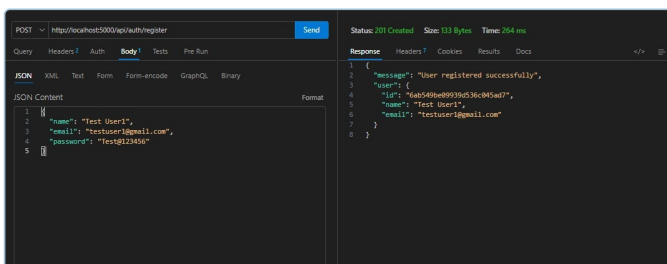
```

Fig. 7 - userModel.js: name (required), email (required, unique, lowercased, trimmed), password (required, minlength 6), with timestamps enabled.

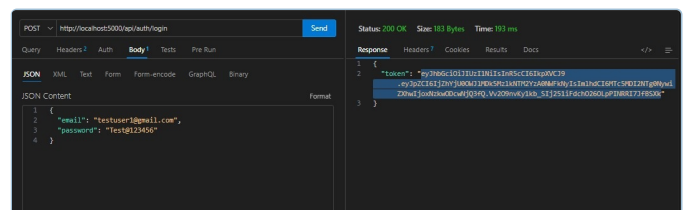
Epic 5: Project Executions

The AI BlogNest backend is executed locally using Node.js while MongoDB handles persistence. Install Postman or the Thunder Client VS Code extension, then start the backend server with `npm start`.

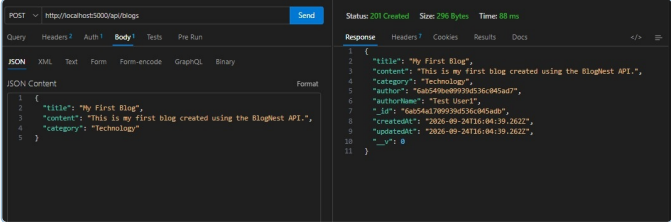
API Testing



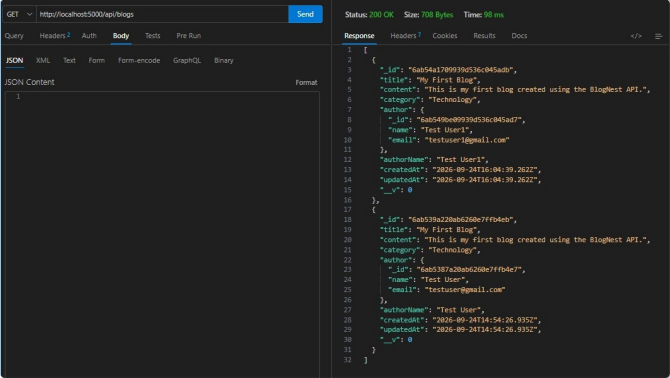
User Registration - POST /api/auth/register, 201 Created in 264 ms, returns the new user's id, name and email.



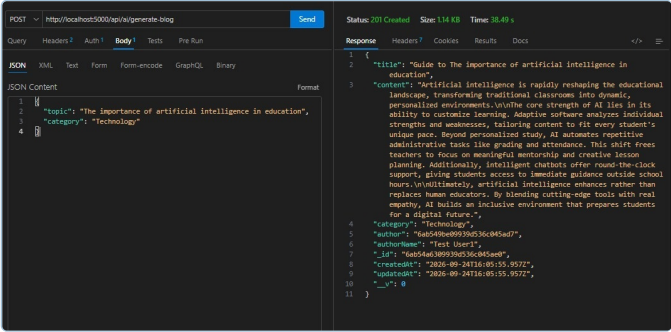
User Login - POST /api/auth/login, 200 OK in 193 ms, returns a signed JWT.



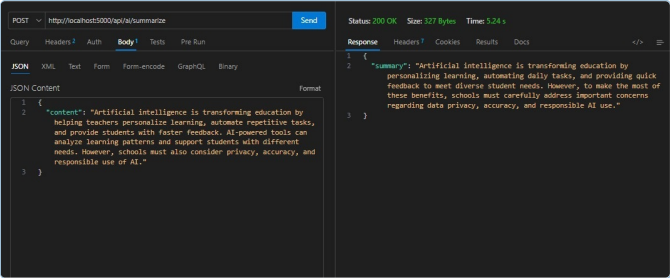
Create Blogs - POST /api/blogs , 201 Created in 88 ms: title "My First Blog", category "Technology".



Fetch Blogs - GET /api/blogs , 200 OK in 98 ms, returning the post list with populated author name & email.



Generate AI blogs - POST /api/ai/generate-blog , 201 Created in 38.49 s for topic "The importance of artificial intelligence in education".



Summarize Blogs - POST /api/ai/summarize , 200 OK in 5.24 s, returning a concise 3-sentence summary.

See Performance Testing for the consolidated results table drawn from this evidence.

Demo & Repository

Demo Video Link	Sivaraj_TN_Skills_V_Sem_Demo_Video - Google Drive
GitHub Link	https://github.com/sivaraj0827/TNskills_AIblognest

Conclusion

AI BlogNest (AI BlogNest API) is a secure, role-based Node.js/Express/MongoDB backend that combines traditional blog and comment management with Generative AI. Built across two focused sprints between 15 and 29 September 2026 by Team SWTID-2026-3342, the project covers authenticated, role-aware user management (Admin/Editor/Author/Reader), full blog CRUD with drafts/categories/tags, comment moderation, and Gemini-powered blog generation and summarization — all verified through Postman testing against a live MongoDB instance. Looking ahead, the team's Scalability & Future Plan extends the data model toward a dedicated Categories collection, richer post metadata (slug, excerpt, cover image) and a likes/reactions layer, positioning AI BlogNest to grow from a focused content-management backend into a fuller publishing platform.